Draw — Project Spec (Cal Hacks 13.0)

Oct 23, 2026 · @Cain

What Draw is

Draw is an AI accounts-payable clerk for small general contractors in which the model can only propose a payment; a deterministic policy kernel, a calibrated decision layer, and a hardware-held signing key decide whether money actually moves.

One-breath version for the team: an AI bookkeeper for construction that refuses to put a subcontractor on the pay request unless the paperwork, the math, the bank account, and the job site all check out, and we build the attacker too, so the demo is "try to steal from us."

Who it's for. The owner or office manager at a 10–30 person commercial GC doing $5–25M projects, who compiles every subcontractor's monthly pay request into the G702/G703 they sign and send to the architect and lender. Their name is on it. If a sub over-billed or a lien release is missing, it's their reputation and their money.

What it does, in order:

1. Reads each sub's pay request (AIA G702/G703 format), lien release, insurance certificate, and any email from the sub.
2. Checks the claimed percent-complete against the super's site video.
3. Runs every rule the GC runs by hand today: retainage math, release present and correct, insurance current, billed-to-date within contract plus approved change orders, lower-tier suppliers released, owner's draw funded, bank details unchanged, vendor verified.
4. Decides release / hold / escalate, with a plain-English reason and the evidence.
5. Requires a physical tap on a device the owner holds before any large or unusual payment goes out.
6. Logs every fact and decision so the whole month can be replayed.
7. Ships with a red team that fuzzes thousands of fake pay requests and reports exactly how often the system can be robbed.

What's different from every other "AI CFO" at this hackathon. The LLM never touches money. Three layers sit between the model and the bank, and the model is the least trusted one. We can show a number for "how often does it pay the wrong person" because

we built the attacker. And we talked to a GC who has lived every failure mode in this doc.

The problem: how a small GC pays subs today

Everything below comes from a 10/23 interview with the owner of a small commercial GC (projects $5–25M each). He runs this process himself because he doesn't trust anyone else with it.

The monthly cycle.

1. Subcontracts require every sub to submit a pay request by the 25th for work completed to date.
2. The GC verifies each sub's request. Supervisors walk the site and comment on whether the claimed percent-complete is real.
3. The office manager confirms each sub attached a lien release and a current certificate of insurance (including workers' comp). No release or no insurance means the sub is left out of this month's request and doesn't get paid until next month.
4. The GC compiles all sub requests, plus his own overhead and profit, into a G703 (the line-item continuation sheet) which populates the G702 (the summary page). These are American Institute of Architects standard forms.
5. The architect reviews and signs off.
6. The lender sends an inspector who walks the project with the pay request in hand and checks every line against what's built.
7. The lender funds the draw. The GC pays subs by the 10th.
8. A conditional lien waiver comes with the request; an unconditional waiver comes after payment clears, about a month later.

The numbers. One pay request per project per month. 40–60 line items per request, each tied to a subcontractor. Every line is cumulative: this month's amount is total-to-date minus everything previously billed.

The part he dreads. A sub requesting money for work they didn't do. He has to catch it before he signs, because his integrity with the architect and lender is on the line. Then he has to tell the sub no, which turns into an argument. This recurs every month.

Pay-when-paid is nearly absolute. He pays a sub before the owner's draw lands "very rarely," and only when he is himself the owner and can negotiate.

What he already does that we can use. He GoPros everything before drywall. Supers video the site at multiple points. Files live in Dropbox, one folder per project, broken down by permits, subs, and original proposals. The portfolio lives in Excel and Google Sheets where everyone works live. He records Zoom meetings with AI. None of this is in a construction software product.

What it costs

Two stories from one GC.

- **$1.5M wire, almost gone.** A sub's email was duplicated with a capital I in place of a lowercase l in the domain, and a wire to new bank details was requested. Nobody in his office caught it. The person processing the wire at the bank did. That is the only reason the money is still there.
- **$70K lien and a fraud exposure.** A sub went out of business. The sub had provided a release, but their concrete supplier had filed a notice to owner and had not released. The office manager missed it. The GC had already signed an affidavit that everyone was paid, which is fraud if it's wrong. He came within reach of losing $70K and found the money himself.

The recurring cost. Every month: verifying 40–60 cumulative line items per project against the site, arguing with subs who over-billed, and chasing releases and insurance certificates. He does this personally because the alternative is trusting someone else with his signature.

The industry cost. Construction is the most-targeted sector for payment-diversion fraud because it has long supply chains, frequent high-value payments, and payment instructions that travel by email.

Figure	Source
Construction and manufacturing were 25% of UK invoice fraud in 2024/25, more than any other sector	NCA / NFB campaign, March 2026
Invoice fraud was 85% of all payment-diversion losses in Sept 2025; avg loss over £47K per case	Project Scotland, March 2026
BEC made up 76% of cyberattacks on construction companies (Kroll)	Vector Choice

Figure	Source
FinCEN: manufacturing and construction are the top BEC targets; BEC took over $300M/month from US firms in 2018	Security Boulevard, ABA Banking Journal
Canonical attack: spoofed sub email claims updated banking details and sends a new invoice	AFP, 2026

Lien exposure is the quieter cost. Paying a sub without a valid release from the sub and its suppliers means the owner's property can be liened for work the GC already paid for. The GC pays twice or litigates. Small GCs track this in a spreadsheet, by one person, under deadline pressure.

Why existing tools fail, and the design rule that follows

Procore (what he tried). He bought it the year it launched, at a national builders' show. The price roughly tripled to about $10K once they were committed. The bigger problem was human input: his people ended up on the computer instead of on the site checking work physically. Trades are aging; adoption and computer literacy were the weak link. He dropped it.

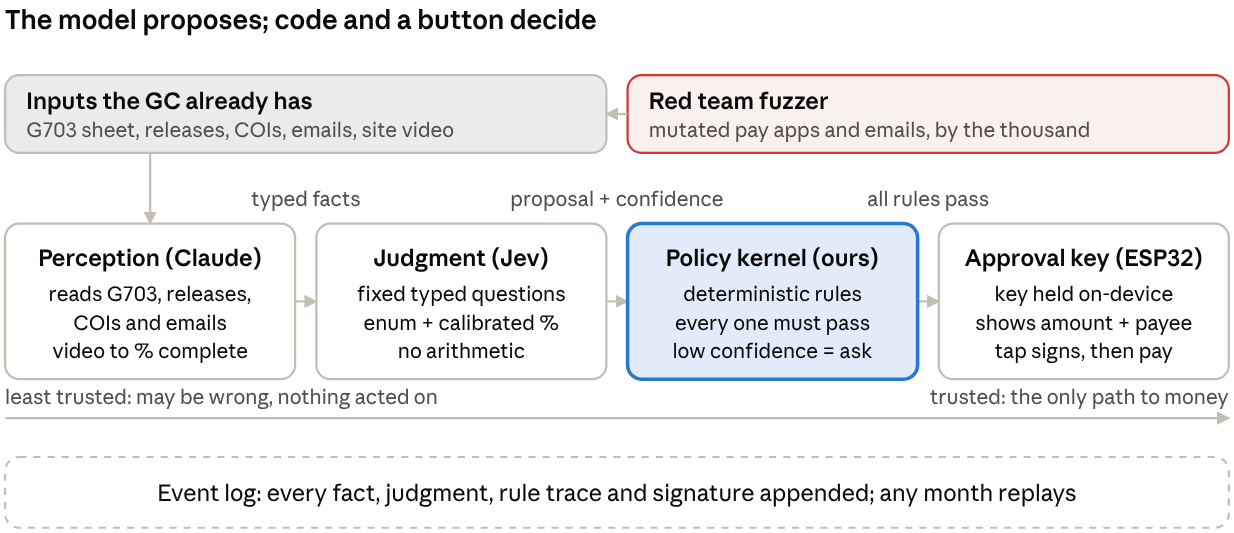
Mid-market AP tools (Procore Pay, GCPay, Levelset). Built for firms with a controller and an IT budget. They track waivers well but assume someone will key everything in.

AI bookkeeping agents (the dozen that will be at this hackathon). They put a language model directly in the payment path: LLM reads the invoice, LLM decides, code pays. Nobody can say what they do on an input they haven't seen, and nobody can prove they won't pay the wrong account. For a company signing lien affidavits, that's the wrong architecture.

His words on what would make him buy: if it handled all the aspects and didn't slow his people down, $10K wouldn't be a problem. Losing time in the field costs him more than software.

The design rule. Zero new data entry. Draw ingests what already exists: the G703 spreadsheet, the Dropbox project folder, the sub's email, the super's GoPro video. If the demo shows anyone typing into a form, we've rebuilt Procore. The office manager's job becomes approving traces and tapping "ask for the release"; the owner's job becomes pressing one button for big payments.

Architecture



Draw architecture · 4 layers, trust rising left to right

Trust rises left to right. Perception and judgment can be arbitrarily wrong and the company still doesn't lose money, because only the kernel's all-pass trace plus a physical signature can release a payment. The red team attacks the same inputs the real sub does, so the scorecard measures the real pipeline.

Layer by layer.

1. **Perception (Claude, untrusted).** Structured-output extraction from the G703 spreadsheet or PDF, lien releases, certificates of insurance, change orders, and sub emails into a typed fact schema. A vision call over the super's site video (sampled frames) estimates percent-complete per line item. Output is facts, never actions.
2. **Judgment (Jev, bounded).** TypeSafe's System One model takes the fact schema plus a fixed set of typed questions and returns an enum per question with a calibrated probability, in hundreds of milliseconds and for a fraction of a cent. It can't emit an action outside the enum, and it tells us how sure it is. It does no arithmetic; code does the math and hands it the result.
3. **Policy kernel (our code, trusted, deterministic).** Pure rule functions over contract state. Each returns (rule name, pass/fail, evidence). A payment proposal executes only on all-pass. Confidence below the escalation threshold on any Jev question forces escalation regardless of the proposed action. Thresholds, spend limits, and never-allowed actions live in an editable permissions contract the kernel reads at runtime.

4. **Approval key (ESP32, physically gated).** Over-threshold payments, first payments to a new sub, and any payment after a bank-detail change require an ECDSA signature from a key generated on the device. The device displays amount, payee, and job; a human taps; the device signs; the kernel verifies the signature before executing. A compromised server or jailbroken model has no path to the key.
5. **Event log.** Append-only. Every fact, judgment, kernel trace, signature, and execution. Replay any month from zero; rerun under a changed contract ("what if retainage were 5%?").
6. **Red team.** A mutation fuzzer that drives thousands of adversarial cases through layers 1–3 and reports where money would have moved wrongly. Judges can author their own attack.

What's real vs. simulated (Hyphenate asks). Real: the kernel, the fuzzer, the event log, the hardware signing, the Jev and Claude calls. Simulated: the GC, its subs, the bank (Capital One Nessie sandbox), vendor identities (CRS sandbox test identities), and the site footage (our own photos or his, if he shares some).

Data model

The seed data is one fictional GC, three active jobs, about 15 subs, six months of pay requests, and roughly 25 planted anomalies with ground-truth labels. Build it to the real forms. The GC shared his own G702, G703, and an executed subcontract (Dropbox links in the interview notes); once those are parsed, their exact columns and contract terms replace the standard-form assumptions below.

Pay application (G702 summary + G703 continuation). One per sub per job per month. The G703 is the line-item sheet; the G702 totals it.

G703 column	Meaning	Kernel use
Item no. / Description	One line per scope of work	Join key to the subcontract schedule of values
Scheduled value	Contract amount for this line	Ceiling for billed-to-date (plus approved COs)
From previous applications	Cumulative billed before this month	Prior state; must equal last month's total completed

G703 column	Meaning	Kernel use
This period	Work completed this month	The delta actually being paid
Materials presently stored	Delivered, not installed	Optional; flag if large
Total completed and stored to date	Previous + this period + stored	Cumulative; this is what the % is computed from
% complete	Total to date / scheduled value	Compared to video estimate
Balance to finish	Scheduled value minus total to date	Sanity check
Retainage	Usually 10% of total to date	Must match contract rate

G702 fields that matter: original contract sum, net change by change orders, contract sum to date, total completed and stored, retainage, total earned less retainage, less previous certificates for payment, current payment due.

Cumulative billing is the whole trick. There is no "invoice amount." This month's payment = (total completed to date minus retainage) minus everything previously certified. The kernel must carry per-line billed-to-date across months and recompute the delta itself, never trust the sub's "this period" column.

Other entities.

Entity	Fields	Why it exists
Job	owner, lender, architect, contract sum, retainage rate, pay-when-paid flag	Per-job rules
Subcontract	sub, job, schedule of values (line items), retainage %, insurance requirements, release requirements	Source of truth the kernel checks against
Change order	job, sub, line items added or changed, approved (bool), date	Billed-to-date may exceed SOV only by approved COs

Entity	Fields	Why it exists
Lien release	sub, job, type (conditional progress / unconditional progress / conditional final / unconditional final), through date, amount, signed (bool)	No release, no payment; unconditional for last month must arrive
Lower-tier notice	supplier, sub, job, notice-to-owner date, released (bool)	The $70K near-miss
Certificate of insurance	sub, GL expiry, workers' comp expiry, additional insured (bool)	Expired COI = held
Vendor	legal name, EIN, email domain, bank account on file, first-paid date, CRS verification status	Identity and bank-change checks
Bank ledger (Nessie)	owner draws in, sub payments out, dates	Pay-when-paid; duplicate detection
Site evidence	job, date, frames/video, per-line % estimate	Pay-on-proof
Email	from, from-domain, claims extracted (bank change, amount, etc.)	BEC detection

Planted anomalies for ground truth (the fuzzer generates more): over-billed % on 4 lines, retainage slipped back in on 2 requests, missing conditional release on 3, expired workers' comp on 2, billed past SOV without a CO on 3, duplicate request with new number on 2, bank-detail change by email on 3 (one with a homoglyph domain), lower-tier supplier with notice and no release on 2, request submitted before the owner's draw cleared on 2, new sub with mismatched EIN on 2.

Policy kernel rules

Every rule is a pure function `(state, proposal) -> (name, passed, evidence)`. The runner executes all of them and releases only on all-pass; the ordered list of results is the audit trace. About 12 rules, 200–300 lines of Python. No policy engine (OPA, Cedar); the rules need arithmetic over our own state and a hand-written kernel is the only thing a judge can read in two minutes.

#	Rule	Check	Evidence returned	From the interview
1	Cumulative continuity	Each line's "previous applications" equals last month's "total to date" in our ledger	Both numbers, per line	Cumulative billing
2	Scheduled-value ceiling	Total to date per line <= scheduled value + approved change orders for that line	Line, SOV, CO sum, overage	Over-billing is his dread
3	Retainage math	Retainage = contract rate x total to date, cumulative; none released before final	Rate, expected, claimed	Standard; 10% typical
4	Conditional release present	A signed conditional progress waiver for this sub, this job, through this period's date	Release id, through date	No release, not in the request
5	Prior unconditional received	Last month's payment has its unconditional waiver on file	Prior payment id, waiver id	Conditional now, unconditional a month later
6	Insurance current	GL and workers' comp expiries after the payment date; GC listed as additional insured	COI id, expiry dates	Office manager tracks this every request
7	Lower-tier released	Every supplier with a notice to owner on this sub/job has a release on file	Supplier, notice date, release status	The $70K miss
8	Owner draw funded	Nessie ledger shows the owner's draw for this period cleared	Transaction id, date, amount	Pay-when-paid, near-absolute

#	Rule	Check	Evidence returned	From the interview
9	Vendor verified	CRS sandbox identity check passes; EIN and legal name match the subcontract	CRS response id	New-sub fraud
10	Bank details unchanged	Account on the request equals account of last successful payment; any change requires out-of-band verification flag set by a human	Old, new, verification status	The $1.5M wire
11	Sender domain authentic	Email from-domain equals vendor domain exactly; confusable-character (homoglyph) distance = 0	Both domains, confusables found	Capital I for lowercase l
12	Not a duplicate	No prior request with same sub/job/period or same line totals under a different number	Matching request id	Standard
13	Progress plausible	Jev's plausibility answer, with the video estimate within the contract's tolerance of the claim	Claimed %, estimated %, gap	Supers verify on site
14	Within spend authority	Amount <= contract's auto-release limit; else escalate to the approval key	Limit, amount	What he'd let a new hire pay alone

#	Rule	Check	Evidence returned	From the interview
15	Confidence floor	Every Jev answer above the escalation threshold; else escalate	Question, probability, threshold	Knowing when to ask

Decision logic. All pass and amount under limit: release. All pass and amount over limit, or new vendor, or bank change: escalate to the approval key. Any fail: hold, with the failing rules and their evidence as the reason, and the next action (ask sub for release, request updated COI, call sub at number on file to verify bank change).

Permissions contract (editable, read at runtime).

```
auto_release_limit_usd: 25000
escalation_confidence_threshold: 0.85
progress_tolerance_points: 10
never_allowed:
  - change_vendor_bank_details
  - release_retainage_before_final
  - pay_before_owner_draw
always_escalate:
  - first_payment_to_vendor
  - bank_detail_change
  - amount_over_limit
```

Change a number in the UI, watch the next decision change. This is the literal answer to Hyphenate's "which seats, with what access."

Judgment layer: Jev questions

Jev (TypeSafe AI, early access since Sept 15 2026) takes a state object plus declared typed questions and returns an enum value and a calibrated probability per question, in roughly 70–500 ms. It generates no text and does no arithmetic. We compute every number in the kernel first and pass Jev the result; Jev supplies the judgment calls a human would otherwise make by eye.

Question	Type	State passed in	What the answer feeds
Is this release the correct form for this payment type?	Choice: correct / wrong_type / incomplete	Release type, payment stage (progress/final), through date, signature present	Rule 4
Does this bank-change request look like account takeover?	Score 0–1	Sender domain vs vendor domain, days since last contact, urgency language present, prior bank changes	Rule 10 escalation reason
Is the claimed progress plausible?	Choice: plausible / overstated / understated	Claimed %, video-estimated %, last month's %, days elapsed, crew size if known	Rule 13
Is this request a duplicate of a prior one?	Score 0–1	Line-total similarity to prior requests, number pattern, submission gap	Rule 12
Proposed action	Choice: release / hold / escalate	Summary of rule inputs	The proposal the kernel checks

Why Jev and not Claude structured output for this layer. Calibration and cost. A calibrated probability turns "escalate when unsure" into a number the GC can set. And at Jev's price we can run the fuzzer with the model in the loop over thousands of cases, which we could not afford with a frontier LLM. Claude stays where it's needed: extraction from messy documents and the vision estimate (Jev is text-only).

Confidence threshold. Start at 0.85. After seeding, run split conformal prediction on the labeled anomalies to set the threshold so the wrongful-release rate is bounded (target <= 1% at 95% coverage). That makes the threshold a statistical statement, not a tuned guess, and the calibration plot goes on the scorecard.

Fallback. Direct Jev access is waitlisted; no-waitlist access is via Vercel AI Gateway (`typesafe-ai/jev`) or Cloudflare Workers AI (`typesafe/jev`). If neither works by Friday night, swap this layer for Claude structured output with self-reported confidence and keep the same question schema. Two hours lost, nothing else changes.

Sources: awesome-jev wiki, Hands-on guide to Jev, SiliconANGLE launch coverage.

Hardware approval key (ESP32)

The approval button that sits on the owner's desk. Big or unusual payments don't go out until he physically presses it. In production this is a push to his phone; at the hackathon the ESP32 makes the separation visible and un-fakeable, and qualifies for Best Hardware Hack.

Flow.

1. On first boot the ESP32 generates an ECDSA P-256 keypair (mbedTLS, in the ESP-IDF). The private key never leaves the device. The public key is registered with the kernel once.
2. When the kernel decides "escalate," the server POSTs the payment payload (payee, amount, job, request id, nonce) to the device over WiFi.
3. The device displays amount, payee, and job (on-board screen if the rental board has one; otherwise it serves a one-page approval screen over WiFi from its own IP, same mono font as the dashboard, amount huge, one green button, one red).
4. A human taps approve. The device signs SHA-256(payload) and returns the signature. Reject returns a signed rejection.
5. The kernel verifies the signature against the registered public key, then executes the Nessie payment and appends signature plus payload to the event log.
6. Nonce and 90-second expiry prevent replay. A payment with no valid signature is impossible by construction; the executor only runs on a verified signature.

Constraints. No soldering in the venue: bare dev board, USB power, WiFi on the venue network ("Cal Hacks 13.0" / "hackers!"). Rental booth opens 6 PM Friday; parts back by 2 PM Sunday. Convex's prize includes a Waveshare ESP32-S3 touch display, which is the ideal board if anyone has one.

Why judges will care. Hyphenate's "limited access" and "knowing when to ask" become a physical object. The red-team demo ends with the judge holding the button while a fraudulent $38K release waits on it.

Red team and scorecard

The system ships with its own attacker, because "proof it works" is a Hyphenate criterion and nobody else will have a number for it.

Mutators (each produces a labeled adversarial case from a clean request).

Mutator	What it changes	Rule that should catch it
Bump % complete on one line by 10–40 points	G703 total to date	2, 13
Reset "previous applications" lower than ledger	G703 previous column	1
Reduce or omit retainage	G703 retainage	3
Drop the conditional release, or attach last month's	Release set	4
Expire the workers' comp date	COI	6
Bill a line past SOV with no approved CO	G703 + CO set	2
Add a supplier notice with no release	Lower-tier set	7
Date the request before the owner's draw clears	Request date vs ledger	8
New sub with EIN of a known bad vendor	Vendor record	9
Email requesting new bank details	Email + vendor	10, 11
Same email with a homoglyph domain (l for l, rn for m, 0 for o)	Email from-domain	11
Resubmit a paid request under a new number	Request set	12
Combine any two of the above	Multiple	Multiple

Runner. Generate N cases (target 5,000), push each through perception (mocked for speed where the mutation is structural; real where it's a document), judgment, and the kernel. Record the decision, the trace, and the ground-truth label.

Scorecard (live on the dashboard).

- Wrongful releases: cases where money would have moved on a mutated request. Target: 0. Per-rule this is provable, and that's the headline number.
- Catch rate: mutated cases held or escalated / mutated cases.

- False-hold rate: clean cases held / clean cases. This matters as much as catch rate; an agent that holds everything gets thrown out in a week.
- Dollars protected (sum of wrongful amounts caught) and dollars delayed (sum of clean amounts falsely held).
- Calibration curve: Jev's stated probability vs. observed correctness, binned. If 80%-confident answers are right about 80% of the time, the escalation threshold means something.
- Attacks that got through in earlier runs and what we changed. Confessing a miss is more credible than claiming perfection.

Judge-authored attacks. The attack console lets a judge pick a mutator and a target request, or type a natural-language attack ("the electrician says the owner told them retainage is waived this month") that Claude turns into a concrete mutation. Hyphenate said they may bring their own scenario; this is us handing them the controls.

Stretch, only if everything above is green by Saturday night. Encode the kernel rules in Z3 and prove the invariants (no release without a matching release form; cumulative retainage never under the contract rate; no release past SOV plus approved COs). Unsat means "proven," not "we didn't find one." Three to four hours for someone who has used Z3; skip otherwise.

Product surface

The user is an office manager who runs the company from QuickBooks, Excel, and a phone. The technical depth stays, one tap below a surface that reads like a to-do list. Judges get both.

Three views.

1. **Pay requests.** A list grouped by job. Each row: sub, amount this period, one status chip (Ready / Needs your OK / On hold), and the reason in one plain sentence. Tap a row: the checklist in English (Release: signed through Oct 31. Retainage: 10%. Progress: claimed 70%, video shows about 45%. Bank details: changed since last payment), the documents and video frame side by side, and the next-action buttons (Ask sub for release. Request updated COI. Call to verify bank change, number on file shown). A "Show details" link expands the raw kernel trace in monospace for the judge.
2. **Approve.** The owner's button. The ESP32 screen or its served page: amount huge, payee, job, green and red. Nothing else.

3. **Security.** Behind a tab the office manager never opens: the scorecard, the calibration curve, the attack console, and the permissions contract editor.

Copy rules. Every status is a sentence, not a code. Not `HOLD: RULE_7_FAIL`; "On hold: Coastal Concrete's supplier filed a notice to owner and hasn't released." Every hold comes with the next action as a button. One teammate owns the copy.

Look. Light theme, warm neutral background, one accent. Green pass, amber hold/escalate, red fail, gray pending; color carries nothing else. Big type, big targets, works on a phone. Dark terminal styling only on the Security tab. Show the document and the video frame whenever something is held; people trust what they can see.

Stack. Next.js (App Router) + TypeScript + Tailwind + shadcn/ui; Recharts for the scorecard; FastAPI backend with the kernel, fuzzer, and event store (SQLite); Server-Sent Events from the event log so decisions appear live while the fuzzer runs; Geist Sans for UI, Geist Mono for traces and amounts. Deploy on Vercel (also the no-waitlist route to Jev via AI Gateway). Scaffold with v0 or Claude Code against shadcn Friday night; Saturday goes only to the three views and the device page.

Demo script

Two minutes, demo first, built from the GC's own stories. The demo video opens with scene 2.

1. **The list.** Three jobs, this month's requests, most green. "This is what the office manager sees on the 26th."
2. **Over-billing ($—, the recurring one).** Tap the held request from the drywall sub: claimed 70% on a line, super's video from the 22nd shows about 45%. The frame sits next to the line item with the gap circled. "Our GC catches this himself every month because his name goes on the request to the lender. Now the software catches it before he looks."
3. **The wire ($1.5M).** An email from "the electrical sub" with new bank details. The two domains side by side, the capital I highlighted. Hold, with "call the number on file" as the button. "The GC we interviewed only kept $1.5 million because a bank employee happened to notice this letter."
4. **The lien ($70K).** A request that passes everything, except the concrete supplier filed a notice to owner and never released. Hold. "He signed an affidavit that everyone was paid. That's fraud if it's wrong. His office manager missed this one; the kernel can't."

5. **Judge's turn.** Open the attack console. "Pick one, or type your own." Run it. Show the trace and which rule fired.
6. **The button.** A clean request over $25K. Everything passes; the ESP32 lights up with the amount and payee. Hand the judge the device. They tap. Payment executes; the event log appends the signature. "The server can't do that without him."
7. **The number.** Security tab: 5,000 fuzzed requests, 0 wrongful releases, catch rate, false-hold rate, dollars protected, calibration curve. "That's how often this thing can be robbed. Nobody else here will show you that number."

Hyphenate's one line on seats: "Draw fills the pay-request verification seat at a small GC: the person who decides what goes on the G703 the owner signs."

What's real / what's simulated, said out loud: real kernel, fuzzer, log, signing, model calls; simulated company, bank, identities, footage.

Stage notes. Rehearse Sunday morning. Test the device on venue WiFi Saturday night. Test the laptop under venue lighting. Keep a recorded fallback of scene 6 in case the board drops WiFi.

Prize tracks

Track	Prize	What it needs from us	Status
Hyphenate: Autonomous Systems for Finance	$8K / $5K / $3K; expedited interviews top 5	Realistic data, named seat with limited access, something new, the moment; demo video opening with the moment; live demo	Primary
CRS Financial Intelligence	$1K best overall, $500 financial agent, interviews for identity/fraud	CRS sandbox central to vendor verification (rule 9), working live	Core
Capital One Nessie	$300/person	Nessie as the GC's bank ledger (rule 8, payments out)	Core
Best Hardware Hack	Bambu Lab A1 Mini	The ESP32 approval key	Core

Track	Prize	What it needs from us	Status
Anthropic	TBD	Named community (owners and office managers at 10–30 person commercial GCs), real problem from someone who lived it, one thing that works, Claude doing something that matters (extraction, vision) and knowing where it fails	Free
Microsoft: Built for the Real World	Surface laptops, FDE mentorship	Customer understanding, measurable usefulness, path to production, responsible design, end-to-end	Free with a good writeup
Most Technically Complex	NuPhy keyboard	Kernel + fuzzer + conformal threshold + hardware signing	Possible

Not pursuing: Factory, Convex, Spacetime, Fetch.ai, Coinbase, Visa, Apify, Elastic, and every other SDK. Each one costs hours and reads as stapled on. The only exception is Convex if a teammate already knows it cold and wants real-time sync for free.

Team split and build order

Four people, four components, one owner each. The kernel and the fuzzer produce the numbers for the demo; protect their time.

Owner	Component	Friday night	Saturday	Saturday night
A (strongest engineer)	Kernel + DSL + permissions contract	Rule functions 1–8 against hand-made fixtures	Rules 9–15, runner, event store, escalation logic	Z3 stretch only if everything is green
B	Seed data + fuzzer + scorecard	Schema, 3 jobs, 15 subs, 6 months of G703s, 25 labeled anomalies	Mutators, runner, metrics, calibration + conformal threshold	Run 5,000 cases, write the scorecard, record misses
C	Perception + judgment	Claude extraction schema for G703/release/COI/email; Jev access via gateway	Vision estimate from sampled video frames; all 5 Jev questions wired; fallback ready	Tune question state; calibration data to B
D	Dashboard + ESP32	Scaffold Next.js/shadcn; rent the board at 6 PM; keypair + signing endpoint	Three views; copy; SSE from event log; device approval page	Demo video; Devpost draft submitted

Build order (strict). Seed data + kernel → fuzzer → perception → dashboard → pay-on-proof → hardware key → Z3 and conformal only if everything before is green.

Cut order if behind. Z3 first, then the natural-language attack author, then the vision estimate (fall back to a hard-coded video % for the demo line), then the ESP32 (fall back to a phone-served approval page). Never cut the kernel, the fuzzer, or the scorecard.

Not building. Daily job reports and weather logging (the GC asked; it's a different product). Any chat interface. Multi-agent anything. A landing page. More sponsor SDKs. Mobile beyond responsive layout.

Checkpoints.

- [ ] Friday 11 PM: kernel runs rules 1–8 on fixtures; seed schema written; board rented; Jev call succeeds or fallback chosen
- [ ] Saturday 2 PM: fuzzer produces a scorecard on 500 cases; extraction works on a real G703; dashboard shows the list
- [ ] Saturday 8 PM: all four demo holds reproduce end to end; device signs a payment
- [ ] Saturday 11 PM: demo video recorded; Devpost draft submitted
- [ ] Sunday 8 AM: final submission; rehearsed twice; at the table by 10:30

Open items

- [ ] Parse the GC's real G702, G703, and executed subcontract (Dropbox blocks automated download; download and drop them in the chat). Replace the standard-form columns above with his, and pull the retainage rate, insurance requirements, and release language into rules 3, 4, and 6.
- [ ] Jev access: does anyone have a key, or go through Vercel AI Gateway / Cloudflare Workers AI tonight. Decide by 11 PM.
- [ ] Ask the GC for 30 seconds of real site video (before-drywall GoPro) to use in scene 2, and permission to quote him anonymously ("a commercial GC doing $5–25M projects").
- [ ] Which rental board: does it have a screen? If not, the device serves its approval page over WiFi.
- [ ] Confirm the lien-release form set to encode. His "notice to owner" language suggests Florida; California uses four statutory forms. Pick one state and name it in the demo.
- [ ] CRS sandbox: which test identities map to a verified vendor, a mismatched EIN, and a flagged entity. Get keys at the booth or the signup link.
- [ ] Nessie: create the GC's account, seed six months of owner draws and sub payments.
- [ ] Name. "Draw" is a working title (a draw is a progress payment). Decide before Devpost.